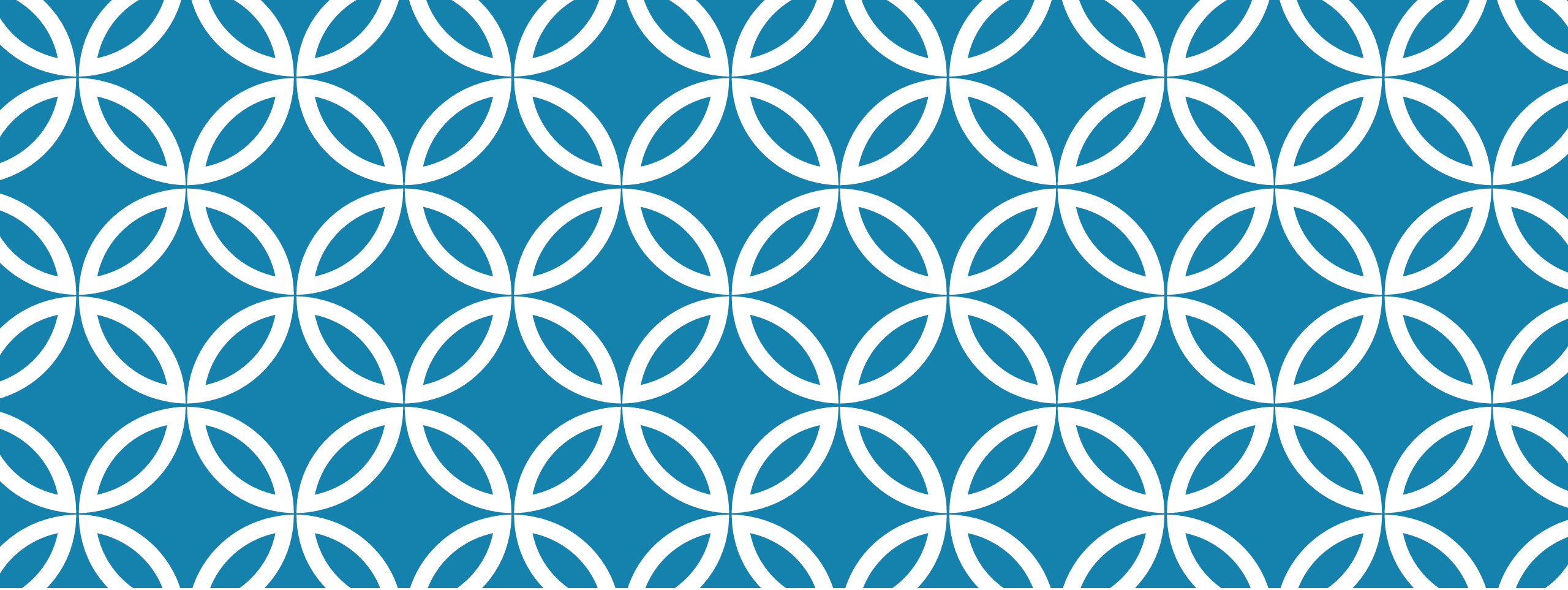


SLR PARSER

SLR PARSER

Bottom up parser

Simple LR parser

The steps will be same as LR(0) parser

The only difference is that the **reduce action** is placed in the **follow** of the non-terminal

EXAMPLE – 1

S → XX

X → aX | b

Check if this grammar can be parsed using SLR parser

We will need to find the follow set for the non terminals

	First	**Follow**
S:	{a,b}	{$}
X:	{a,b}	{a,b,$}

(1) Augment the grammar

S → S’

S → XX

X → aX

X → b

(2) Number the production

0: S → S’

1: S → XX

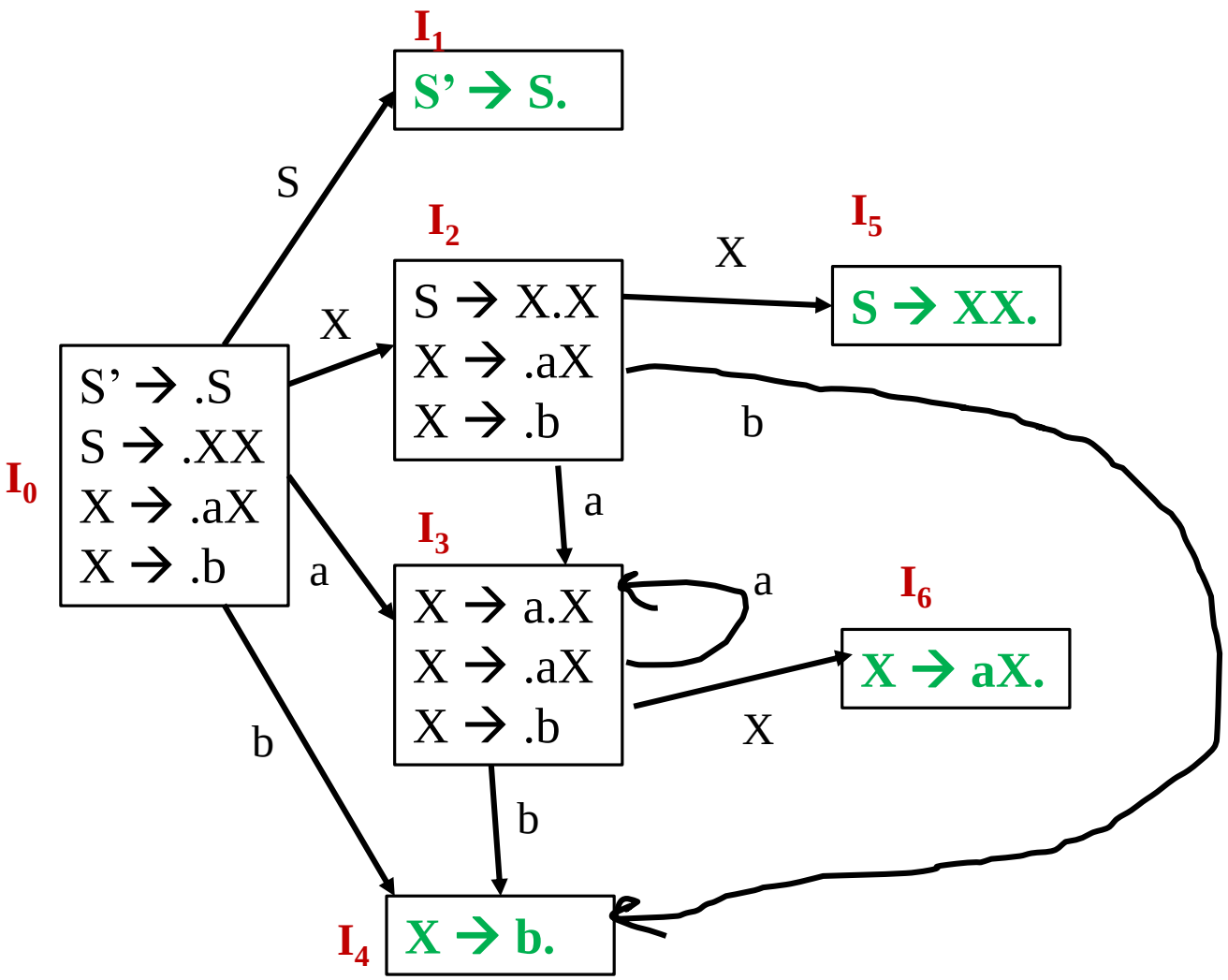
2: X → aX

3: X → b

	First	Follow
S:	{a,b}	{$}
X:	{a,b}	{a,b,$}

0: S' → S
1: S → XX
2: X → aX
3: X → b

(3) Find the LR(0) canonical items



(4) Create the SLR Parser table

States	Action			Goto	
	a	b	$	X	S
I_0	S_3	S_4		2	1
I_1			Accept		
I_2	S_3	S_4		5	
I_3	S_3	S_4		6	
I_4	r_3	r_3	r_3		
I_5			r_1		
I_6	r_2	r_2	r_2		

No Shift-reduce or Reduce-reduce conflict
Hence grammar can be parsed using SLR parser

EXAMPLE – 2

$A \rightarrow (A) \mid a$

Check if this grammar can be parsed using SLR parser

TRY THIS OUT

E → T + E | T

T → T * V | V

V → id

Check if this grammar can be parsed using SLR parser

EXERCISES

(1) $S \rightarrow aAB \mid bA$

$A \rightarrow cB \mid \varepsilon$

$B \rightarrow e \mid \varepsilon$

(2) $S \rightarrow aZ \mid b$

$Z \rightarrow a \mid b$

(4) $S \rightarrow Aa \mid bAc$

$A \rightarrow d$

(5) $S \rightarrow (E)$

$E \rightarrow V \,\&\, E \mid V$

$V \rightarrow !\,(E) \mid a$